

Setting Up Multiple SSH Keys and OAuth2 on Linux

Introduction

When working with multiple systems and repositories, you often need more than one SSH key. You may need one key for connecting to your virtual machine from your laptop, and another for that same virtual machine to connect securely to GitLab. This guide explains how to manage multiple SSH keys, handle existing ones safely, configure OAuth2 authentication for private GitLab instances, and resolve common SSH connection issues such as permission errors.

Managing Access to Your Virtual Machine

Your virtual machine uses the file `~/.ssh/authorized_keys` to determine who is allowed to log in. Each public key in that file grants access to the corresponding private key holder. To allow multiple users or devices, add each key on a new line, for example:

```
ssh-ed25519 AAAAB3NzaC1yc2EAAAADAQABAAQ... mylaptop
ssh-ed25519 AAAAB3NzaC1yc2EAAAADAQABAAQ... coworker
```

Understanding Comments in `authorized_keys`

At the end of each key in `authorized_keys`, there is usually a comment — a label added when the key was created. It helps identify which device or person the key belongs to. For example:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIB1ABCxyz123... mylaptop
```

The text after the key (mylaptop in this case) is purely informational. Removing or changing it does not affect SSH functionality. The key will still work normally, but keeping a meaningful comment is recommended for clarity — it helps you know which key belongs to which device or user.

Creating a Key for Your Virtual Machine

If your virtual machine needs to connect to GitLab, it should have its own SSH key pair. Generate one with the following command:

```
ssh-keygen -t ed25519 -C "vm-gitlab-key"
```

This command creates two files in your `~/.ssh` directory: the private key (`id_ed25519`) and the public key (`id_ed25519.pub`). Keep the private key secure on the VM, and add the public key to GitLab.

When an SSH Key Already Exists

If you already have an SSH key in your `~/.ssh` directory, do not overwrite it when generating a new one. Check which keys exist first:

```
ls -l ~/.ssh
```

If a file such as `id_ed25519` already exists, choose a new name for your next key when prompted by `ssh-keygen`. For example:

```
ssh-keygen -t ed25519 -C "vm-gitlab-key" -f ~/.ssh/id_ed25519_gitlab
```

This creates separate files named `id_ed25519_gitlab` and `id_ed25519_gitlab.pub`, allowing you to maintain multiple key pairs safely. Each key can be referenced individually in your SSH configuration file.

```
Host gitlab.com
    HostName gitlab.com
    User git
    IdentityFile ~/.ssh/id_ed25519_gitlab
```

Repeat this approach whenever you generate new keys for different systems or services. Use descriptive filenames like `id_ed25519_vm`, `id_ed25519_gitlab`, or `id_ed25519_github` to keep your keys organized.

Setting Up OAuth2 for Private GitLab Access

If you are working with a private GitLab instance such as `gitlab.endava.com`, SSH may not be sufficient. You can use OAuth2 client credentials for authentication with Git. Set up the necessary credentials globally using these commands:

```
git config --global credential.https://gitlab.endava.com.gitLabDevClientId
    <APP_ID_FROM_GITLAB>
git config --global credential.https://gitlab.endava.com.gitLabDevClientSecret
    <APP_SECRET_FROM_GITLAB>
git config --global credential.https://gitlab.endava.com.provider gitlab
git config --global --get-urlmatch credential https://gitlab.endava.com
```

Clearing or Resetting Credentials and Selecting a Mode

To remove or reset credentials for another GitLab environment, use the following commands:

```
git config --global --unset-all
    credential.https://gitlab.example.com.gitLabDevClientId
git config --global --unset-all
    credential.https://gitlab.example.com.gitLabDevClientSecret
git config --global --unset-all credential.https://gitlab.example.com.provider
```

Authentication modes:

```
(unset): Automatically detect modes
browser: OAuth authentication via a web browser (requires a GUI)
basic: Basic authentication using username and password
pat: Personal Access Token based authentication
```

To explicitly use browser-based OAuth2 authentication, run:

```
git config --global credential.gitLabAuthModes "browser"
```

For additional guidance, see:

https://docs.gitlab.com/integration/oauth_provider/#create-a-user-owned-application

Fixing Permission Denied (publickey) and Using SSH Config

If you receive an error such as 'Permission denied (publickey,keyboard-interactive)' when trying to clone from GitLab, it means SSH did not use the correct key. This often happens when your key file is not one of the default names SSH looks for automatically.

To test your connection manually, run:

```
ssh -i ~/.ssh/id_ed25519_gitlab -o IdentitiesOnly=yes -T git@gitlab-ssh.endava.com
```

If this command prints 'Welcome to GitLab, @yourusername', your key works. You can then make this configuration permanent by creating or editing your SSH config file:

```
Host gitlab-ssh.endava.com
    HostName gitlab-ssh.endava.com
    User git
    IdentityFile ~/.ssh/id_ed25519_gitlab
    IdentitiesOnly yes
```

Save the file as `~/.ssh/config` and make sure its permissions are correct (`chmod 600 ~/.ssh/config`). After this, SSH and Git will automatically use the correct key when connecting to the Endava GitLab host.

Testing the Setup

After completing your SSH and OAuth2 configurations, test your connection to GitLab:

```
ssh -T git@gitlab-ssh.endava.com
```

If everything is configured correctly, GitLab will confirm the connection. If not, check that your SSH directory and files have the proper permissions:

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/id_ed25519*
chmod 644 ~/.ssh/*.pub
chmod 600 ~/.ssh/config
```

A Note on ed25519 vs rsa

The ed25519 algorithm offers better performance and security with shorter key lengths than rsa. It is modern, fast, and generally preferred for new setups. Only use rsa if older systems require it.

Summary

Your `authorized_keys` file determines who can log into your virtual machine, while your SSH keys and OAuth2 configurations define how that machine connects outward to other systems. By combining SSH with OAuth2, you can securely manage both personal and enterprise GitLab environments. Keep your configuration files organized, your permissions correct, and your credentials private.